

KREACIJSKI PATERNI

-Builder patern

-Builder patern se koristi kada se objekat gradi kroz više koraka i kada konstruktor ima puno parametara. Umjesto jednog složenog konstruktora, Builder omogućava postepeno postavljanje podataka, i to redom koji korisniku odgovara.

-U našem slučaju Builder patern smo iskoristili kod klase Rezervacija.

-S obzirom na to da kreiranje jedne rezervacije zahtijeva kompleksno uvezivanje podataka o korisniku, odabranom terminu, statusu, te proračunu konačne cijene uz popuste, Builder nam omogućava da proces instanciranja učinimo jednostavnijim, preglednim i zaštićenim od grešaka.

-Factory method

-Factory Method patern omogućava kreiranje objekata na način da podklase odluče koju klasu instancirati.

-Factory Method patern odvaja zahtijevanje objekata od njihovog kreiranja.

-U našem sistemu ovaj patern je implementiran unutar podsistema za slanje notifikacija.

-Klasa Rezervacija komunicira isključivo sa apstraktnim kreatorom i interfejsom proizvoda, dok konkretne podklase (EmailNotifikacijaCreator i SMSNotifikacijaCreator) odlučuju koji će tip objekta biti instanciran u pozadini. Time je sistem postao fleksibilan.

-Singleton patern

-Singleton patern osigurava da se klasa može instancirati samo jednom i da osigura globalni pristup kreiranoj instanci klase.

-Također osigurava lazy instanciranje objekata.

-U našem sistemu ovaj patern bismo mogli primijeniti na klasu Popust.

-Klasa Popust u sebi sadrži sva pravila za umanjenje cijena na nivou čitave aplikacije.

-Pravila popusta su ista za sve korisnike.

-Ne treba da postoji više različitih pravila.

-Ažuriranje pravila treba da se vrši na jednom mjestu i da bude odmah vidljivo svima.

-Prototype patern

-Prototype patern dozvoljava da se kreiraju prilagođeni objekti bez poznavanja njihove klase ili detalja kako je objekat kreiran.

-Prototype obezbjeđuje interfejs za konstrukciju objekata kloniranjem jedne od postojećih prototip instanci i modifikacijom te kopije.

-U našem sistemu ovaj patern bismo mogli primijeniti na klasu Termin.

-Kada admin želi generisati raspored termina za cijeli mjesec unaprijed, ručno kreiranje objekata ispočetka bi opteretilo memoriju i usporilo kod(tj. bilo bi resursno zahtjevno).

-Umjesto toga, kreira se jedan bazni termin (sa definisanim terenom, sportom i cijenom) koji se zatim kopira pomoću metode clone().

-Pri kloniranju objekta koristi se plitka kopija (shallow copy).

-S obzirom na to da klasa Termin sadrži samo proste atribute (poput datuma, vremena i cijene), nema kompleksnih ili ugniježđenih objekata, pa je plitko kopiranje sasvim dovoljno i dodatno ubrzava proces.

-Abstract Factory patern

- Omogućava da se kreiraju familije povezanih objekata/produkata bez specificiranja konkretnih klasa.
- Klijent radi sa apstraktnim interfejsima i klasama i ne zna stvarne tipove objekata kreiranih sa fabrikom.
- Patern odvaja definiciju (klase) produkata od klijenta.
- U našem sistemu za rezervaciju , Abstract Factory pattern mogli bismo primijeniti na kreiranje različitih tipova termina.
- U zavisnosti od filtera, konkretne fabrike (PopularniTerminFabrika i NepopularniTerminFabrika) kreiraju odgovarajuće kombinacije elemenata.
- PopularniTerminFabrika kreira skup objekata koji sadrži SkupaCijena i NemaPopusta, dok NepopularniTerminFabrika kreira JeftinaCijena i StudentskiPopust.
- Admin i sistem rade samo sa apstraktnim interfejsima za cijene i popuste, ne zanimaju ih konkretne klase.
- Time se osigurava da sistem za udarni termin (npr. u 20:00h) nikada greškom ne povuče jeftinu cijenu ili pogrešan popust.